STAT 612 Week 6: Midterm 1 (40 pts)

Richard Ressler

2020-02-27

Instructions

- You have two-hours and thirty minutes. There are five pages.
- Put your answers in a new R Markdown document and knit it directly to PDF.
- Submit BOTH the R Markdown document and the knitted PDF to Blackboard before the exam time has expired.
- You do not need to copy and paste each question. Use `#` and `##` tags with the question number for each question.
- You may use your notes, the R help files, and the internet.
- You may not discuss this exam with anyone (including over email, instant message, text, etc. . . .).
- Use the tidyverse, not base R, whereever possible. Too much base R shows less understanding of the material in this class.
- **If you have any questions about what a question is asking you to do please come see me.**
- **Knit your exam after every answer to make sure it is knitting properly.**

Question	**Possible Pts**	**Cumulative**	– **Score** –
1.1	2	1	
1.2	3	4	
1.3	3	7	
1.4	3	11	
1.5	4	15	
1.6	4	19	
1.7	4	23	
1.8	4	27	
2.1	4	31	
2.2	4	35	
2.3	5	40	
EC	3		
	Sub-Total	40	
	End Extra Credit 3		
	TOTAL	40	

Question 1: Star Wars Characters

The `starwars` data frame in the dplyr package contains demographic characteristics of various characters from the hit franchise Star Wars. We will use this data frame for question 1.

1. (2 pts) Load the data into R and load all necessary packages for this exam (if you cannot figure out how to do this, I will tell you since it is required for the rest of this question, but you will not receive credit for this part).

```
library(tidyverse)
data("starwars")
```

2. (3 pts) Use appropriate R code to determine which individuals have missing gender. Print out their names, heights, and species, arranged in ascending order of height.

```
starwars %>%
  filter(is.na(gender)) %>%
  arrange(height) %>%
  select(name, height, species)
```

```
## # A tibble: 3 x 3
##   name  height species
##   <chr>  <int> <chr>
## 1 R2-D2     96 Droid
## 2 R5-D4     97 Droid
## 3 C-3PO    167 Droid
```

3. (3 pts) It seems all individuals in part 2 are droids. Change their gender to `"droid"` and save to the dataframe.

```
starwars %>%
  mutate(gender = replace(gender, is.na(gender), "droid")) ->
  starwars
select(starwars[starwars$gender == "droid",], c(name, height, gender, species))
```

```
## # A tibble: 3 x 4
##   name  height gender species
##   <chr>  <int> <chr>  <chr>
## 1 C-3PO    167 droid  Droid
## 2 R2-D2     96 droid  Droid
## 3 R5-D4     97 droid  Droid
```

4. (3 pts) The body mass index (BMI) is defined as

$$\frac{\text{mass}_{\text{kg}}}{\text{height}_m^2}.$$

That is, the BMI is the weight of the individual (in kg) divided by the height of the individual (in m) squared. There are 100 cm in each m.

Calculate the BMI for each individual in the `starwars` data frame and add it as a permanent variable `bmi` to the `starwars` data frame.

Sanity check: The median BMI should be in the 20-30 range.

```
starwars %>%
  mutate(bmi = mass / (height / 100) ^ 2) ->
  starwars
head(select(starwars, c(name, height, mass, bmi)))
```

```
## # A tibble: 6 x 4
##   name           height  mass   bmi
##   <chr>           <int> <dbl> <dbl>
## 1 Luke Skywalker    172    77  26.0
## 2 C-3PO             167    75  26.9
## 3 R2-D2              96    32  34.7
## 4 Darth Vader       202   136  33.3
## 5 Leia Organa       150    49  21.8
## 6 Owen Lars         178   120  37.9
```

5. (4 pts) Calculate the min and max height for each gender as well as the number of individuals in each gender.

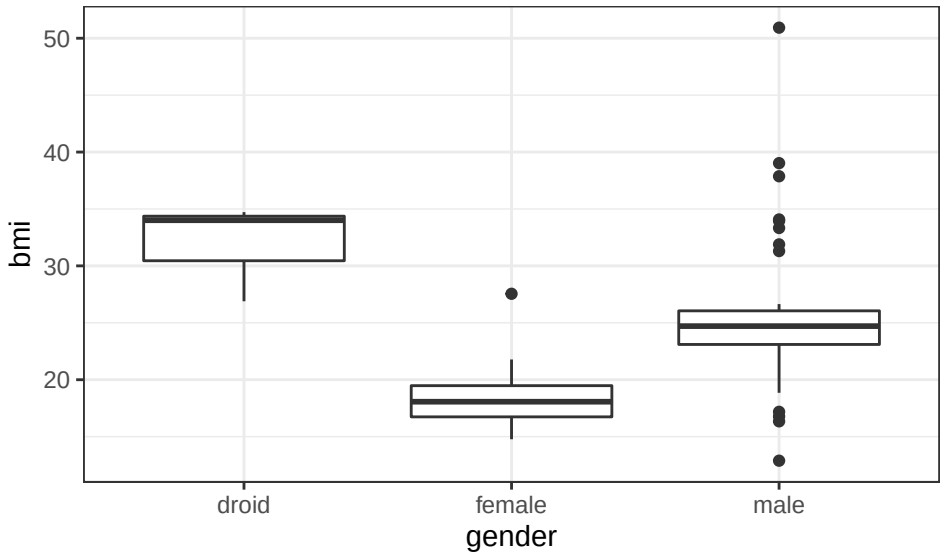
```
starwars %>%
  group_by(gender) %>%
  summarise(min = min(height, na.rm = TRUE),
            max = max(height, na.rm = TRUE), number = n())
```

```
## # A tibble: 5 x 4
##   gender          min   max number
##   <chr>         <int> <int>  <int>
## 1 droid            96   167      3
## 2 female           96   213     19
## 3 hermaphrodite   175   175      1
## 4 male             66   264     62
## 5 none            200   200      2
```

6. (4 pts) Make boxplots of gender vs BMI. Include only males, females, and droids. Use the black and white theme.

```
starwars %>%
  filter(gender == "male" | gender == "female" | gender == "droid") %>%
  ggplot(mapping = aes(x = gender, y = bmi)) +
  geom_boxplot() +
  theme_bw()
```

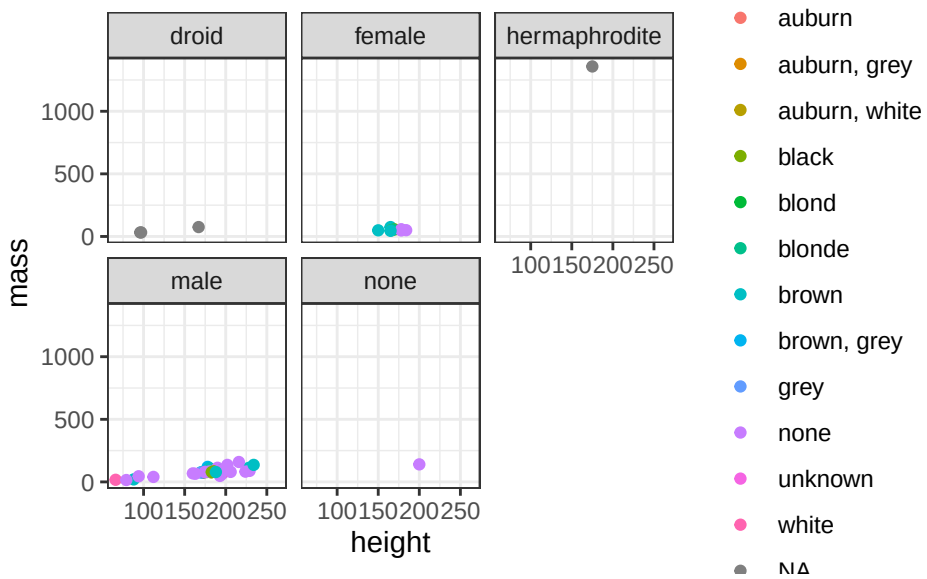
```
## Warning: Removed 27 rows containing non-finite values (stat_boxplot).
```



7. (4 pts) Make scatterplots of height vs mass, differentiated by hair color, faceting by gender.

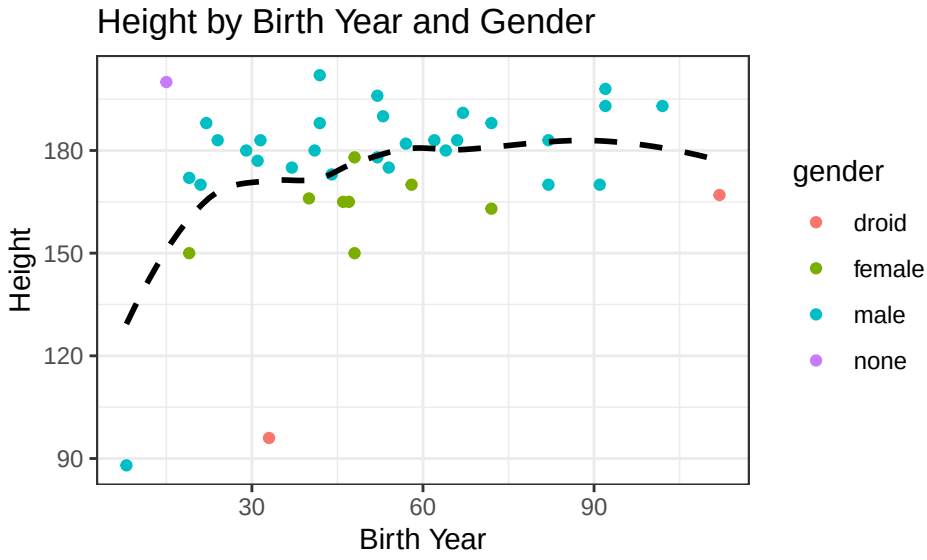
```
ggplot(data = starwars, mapping = aes(x = height, y = mass, color = hair_color)) +
  geom_point() +
  facet_wrap(. ~ gender) +
  theme_bw()
```

```
## Warning: Removed 28 rows containing missing values (geom_point).
```



8. (4 pts) Reproduce the following plot (hint: filter based on birth year):

```
starwars %>%
  filter(birth_year < 200) %>%
  ggplot(mapping = aes(x = birth_year, y = height )) +
  geom_point(aes(color = gender)) +
  ylab("Height") +
  xlab("Birth Year") +
  ggtitle("Height by Birth Year and Gender") +
  geom_smooth(se = FALSE, color = "black", linetype = "dashed") +
  theme_bw()
```



Question 2: Coding Exercises

1. (4 pts) Write a function named `convert()` that takes one number as input and:

- returns the square of an input even whole number,
- returns the cube of an input odd whole number, and
- returns `Inf` for a non-integer input number.

For example, `convert(5)` should give `125`, `convert(8)` should give `64`, and `convert(4.7)` should give `Inf`. Evaluate your function at `0, 4, 7`, and `2.5`.

```r
convert <- function(x) {
  if(x %% 2 == 0) {
    return(x ^2)
  } else if (x %% 2 == 1) {
    return(x ^ 3)
  } else {
    return(Inf)
  }
}
convert(0)
```

```
## [1] 0
```

```r
convert(4)
```

```
## [1] 16
```

```r
convert(7)
```

```
## [1] 343
```

```r
convert(2.5)
```

```
## [1] Inf
```

2. (4 pts) Reproduce the following lines (with URL) and table in R Markdown:

- Typing Test Phrases
  - *The* **quick** brown ~~fox~~ `jumps`

over	the
lazy	dog

- Math Phrases
  - Given a sample of size n from a population with values y_i, we use $\bar{Y}$ to estimate the true mean μ where

$$\bar{Y} = \sum_{i=1}^{n} \frac{y_i}{n}$$

2.3.(5 pts) Write the appropriate R function to do the following:

- Accept an input vector of no more than 5 numbers
  - Stop if the vector has more than 5 values.
- Produce a plot for each value of the input vector
- The plot should be a histogram of random sample of size 1000 from a Normal distribution (mean=input, sd = 1)
  - Use the values of the input vector for the mean.
  - Use 50 bins and color the bars of the histogram with blue edges and a white interior. Use `theme_minimal()`
- Test on the following vectors: c(1,-10.2,5) and c(1,2,3,4,5,6).
- Be sure to set your code chunk option for `error = TRUE`

Hint, assign your plot to a variable and print it each time, e.g.,

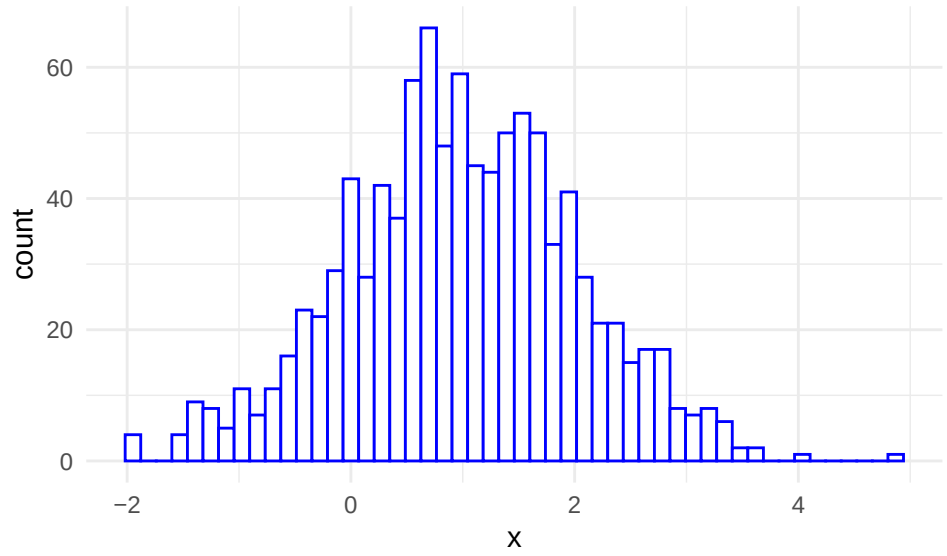
```
p <- ggplot(...) +
geom_histogram(...) +
theme_minimal()
print(p)
```

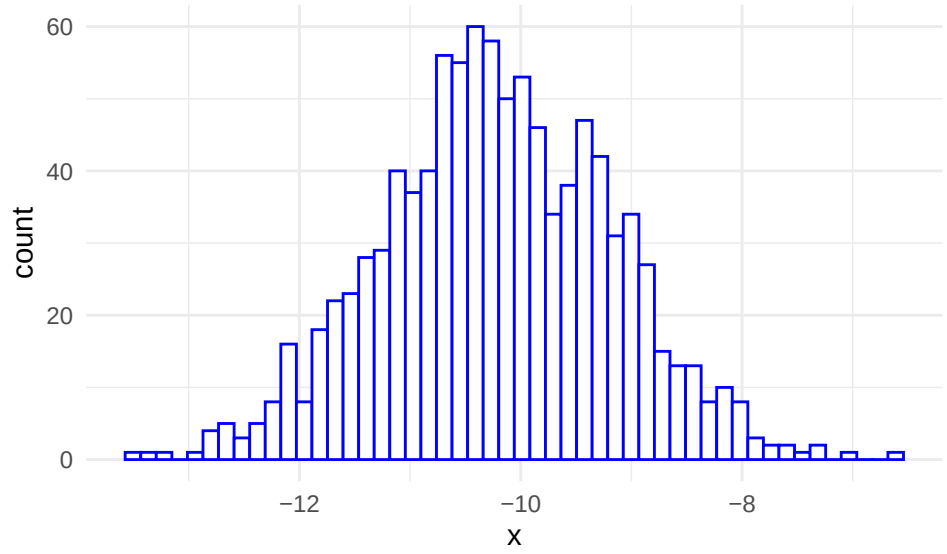
```
library(tidyverse)
set.seed(1)
plot_n <- function(nvec){
  stopifnot(length(nvec)<=5)
 for (i in seq(1:length(nvec))){
   df <- tibble(x = rnorm(1000, mean = nvec[[i]]))
   p <- ggplot(df,aes(x=x)) +
     geom_histogram(fill = "white", color = "blue", bins = 50) +
     theme_minimal()
   print(p)
 } #end for loop

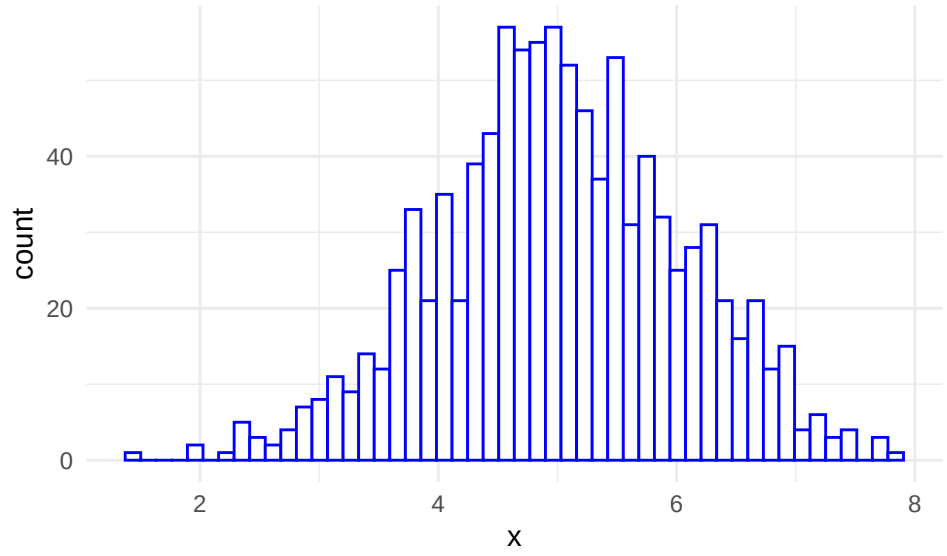
} #end function
```

Your results should look like the following for a seed of 1

```
plot_n(c(1,-10.2,5))
```


60
40
20
0
count
−2
0
2
4
x


60
40
20
0
count
−12
−10
−8
x

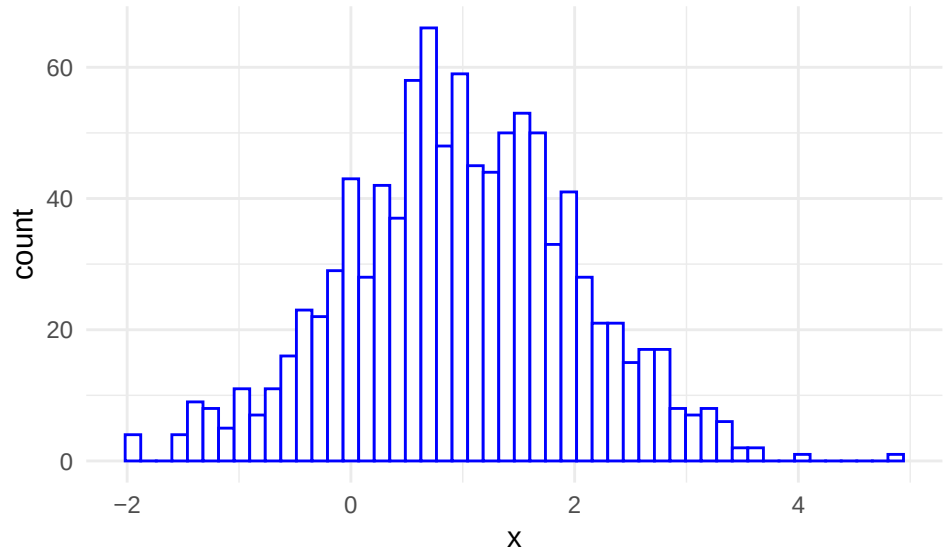

40
20
0
count
2
4
6
8
x

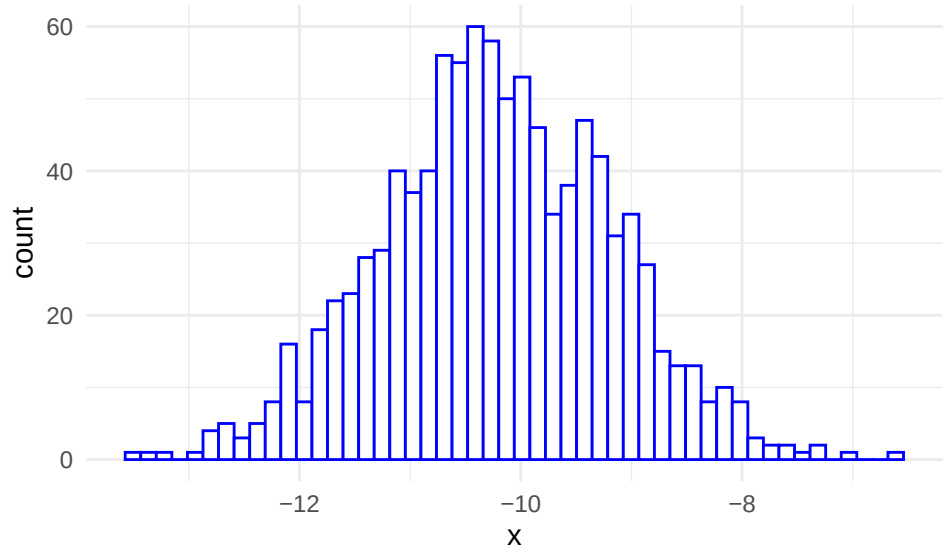
```
plot_n(c(1,2,3,4,5,6))
```

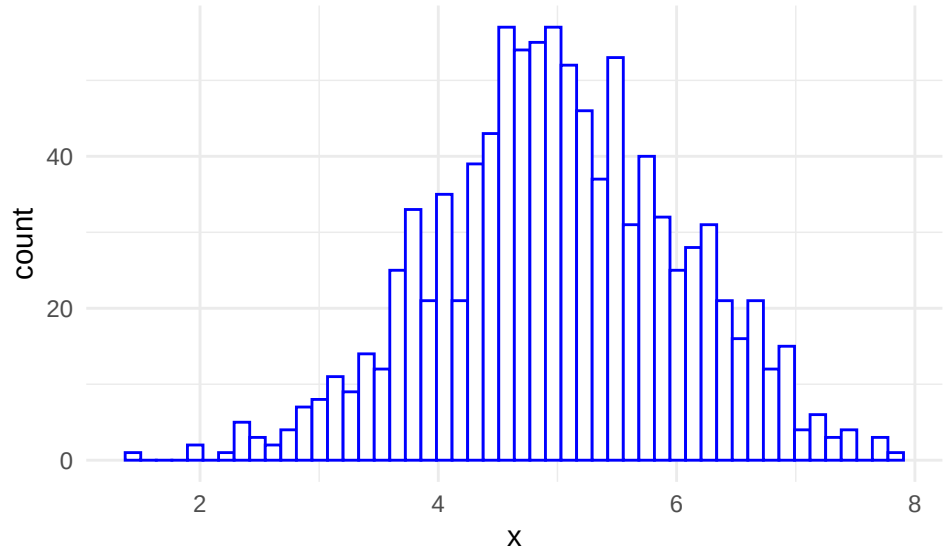
```
## Error in plot_n(c(1, 2, 3, 4, 5, 6)): length(nvec) <= 5 is not TRUE
```

```
set.seed(1)
plot_n2 <- function(nvec){
  lenn <- length(nvec)
  stopifnot(lenn <=5)
 if (lenn == 1) {
     phist(nvec[[1]])
 } else if (lenn ==2) {
phist(nvec[[1]])
phist(nvec[[2]])
}   else if (lenn ==3) {
phist(nvec[[1]])
phist(nvec[[2]])
phist(nvec[[3]])
 }   else if (lenn ==4) {
phist(nvec[[1]])
phist(nvec[[2]])
phist(nvec[[3]])
phist(nvec[[4]])
 } else if (lenn ==5){
phist(nvec[[1]])
phist(nvec[[2]])
phist(nvec[[3]])
phist(nvec[[4]])
phist(nvec[[5]])
 }
} #end function

phist <- function(n){
  p <- ggplot(tibble(x = rnorm(1000, mean = n)),aes(x=x)) +
     geom_histogram(fill = "white", color = "blue", bins = 50) +
     theme_minimal()
   print(p)
}
#phist(3)
plot_n2(c(1,-10.2,5))
```


60
40
20
0
count
−2
0
2
4
x


60
40
20
0
count
−12
−10
−8
x


40
20
0
count
2
4
6
8
x

```r
plot_n2(c(1,2,3,4,5,6))
```

```
## Error in plot_n2(c(1, 2, 3, 4, 5, 6)): lenn <= 5 is not TRUE
```

Question Extra Credit. (3 pts) (do this one last):

- Write a function called `evenodd()` that takes a single positive integer as input. The function should:
  - Print a data frame where the first variable is a sequence, from 1 to the given number, and the second variable indicates whether the value of the first variable in that row is even or odd.
  - Stop if the number is less than or equal to 0 and return a specific error message.
- Evaluate your function at `5`, `8`, and `-100`
  **(in the code chunk options, set `error = TRUE` to allow for errors while knitting).

For example, the following are some evaluations of an implementation of `evenodd()` at 3, 4, 0.

```r
evenodd <- function(n) {
  if (n <= 0) {
    stop("n is less than 1")
  }
  dfdat <- tibble(number = seq_len(n),
                  type = rep(c("odd", "even"), length.out = n))
  return(dfdat)
}
```

```r
evenodd(5)
evenodd(=)
evenodd(-100)
```

```
## Error: <text>:2:9: unexpected '='
## 1: evenodd(5)
## 2: evenodd(=
##            ^
```

```r
evenodd(3)
```

```
## # A tibble: 3 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
```

```r
evenodd(4)
```

```
## # A tibble: 4 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
## 4      4 even
```

```r
evenodd(0)
```

```
## Error in evenodd(0): n is less than 1
```